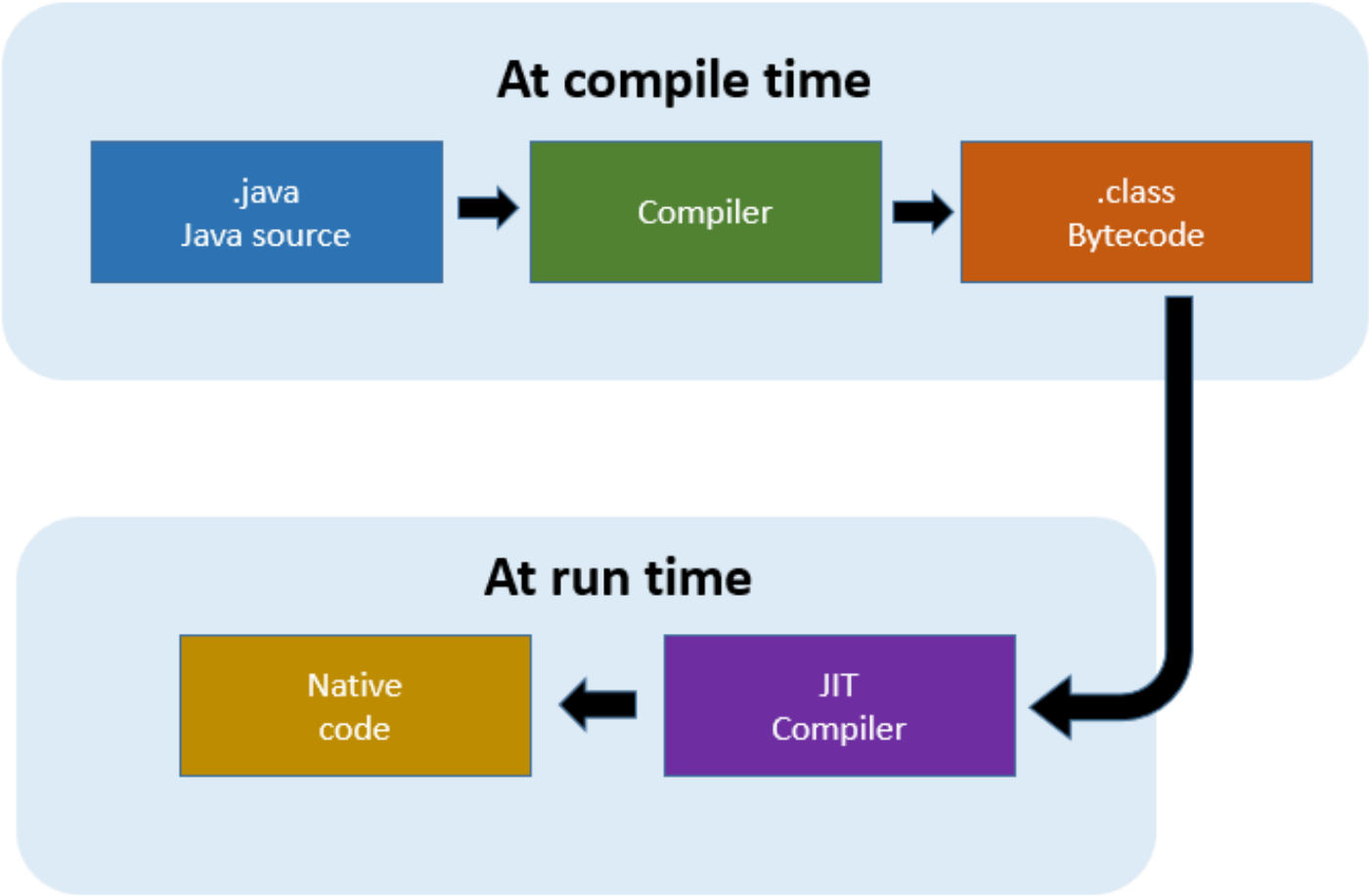
Chapter 11:
Inheritance and Polymorphism
Part-II

11.7. Static and Dynamic Binding

👉Motivation

- ✨Inheritance lets subclasses use methods from their superclass but can lead to confusion about which method gets called.
- ✨The JVM decides which method to run either at compile time or runtime.
- ✨**Binding** is the process of linking a method call to its method definition.
- ✨There are two types of binding:
  - ✨**Static Binding**: Happens at compile time.
  - ✨**Dynamic Binding**: Happens at runtime.

👉Visualization



JIT: Just-In-Time

👉What is Static Binding?

✨**Static binding** (or early binding) is the process where the method to be executed is determined at compile time based on the declared type of the object.

✨**Compile time** refers to the period when the code is being compiled into bytecode, before it is executed.

✨During compile time, the compiler checks the declared type of the object and binds the method call to the method defined in that type.

Example: Static Binding

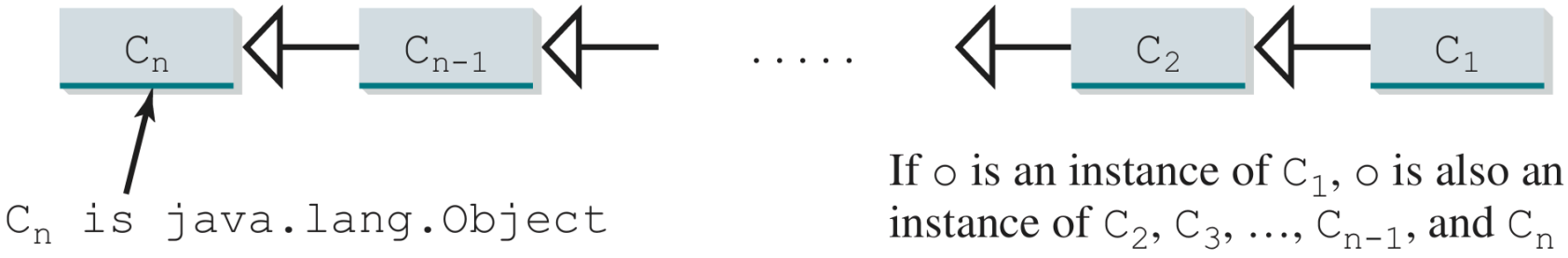
```
class A {
    static void show() { System.out.println("A"); } // Static method
    final void display() { System.out.println("A-final"); } // Final method
}
class B extends A {
    @Override
    static void show() { System.out.println("B"); }
    // Cannot override display()
}
public class Main {
    public static void main(String[] args) {
        A a = new B();
        a.show();      // A (static binding)
        a.display();   // A-final (final method)
        B b = new B();
        b.show();      // B (static binding)
    }
}
```

👉What is Dynamic Binding?

✨**Dynamic binding** (or late binding) is the process where the method to be executed is determined at runtime based on the actual type of the object, rather than the declared type.

✨**Runtime** refers to the period when a program is executing, after it has been compiled.

✨During runtime, JVM determines which method to execute based on the actual type of the object.



Example: Dynamic Binding

```
class A {
    void display() { System.out.println("A"); }
}
class B extends A {
    void display() { System.out.println("B"); }
}
class C extends A {
    void display() { System.out.println("C"); }
}
public class DynamicBindingDemo {
    public static void main(String[] args) {
        A obj1 = new B(); // obj1 is declared as type A but refers to an instance of B
        A obj2 = new C(); // obj2 is declared as type A but refers to an instance of C
        obj1.display(); // Output: B
        obj2.display(); // Output: C
    }
}
```

👉Actual Type vs. Declared Type Object

✨An `actual type object` is the type of the object that the variable refers to at runtime.

Syntax:

```
SuperClass object = new SuperClass();
```

✨A `declared type object` is the type of the object as declared in the code.

Syntax:

```
SuperClass object = new SubClass();
```

✨The `declared type object` can be a superclass, while the `actual type object` is the specific subclass that the object belongs to.

👉 Method Matching vs. Method Binding

✨ Method Matching

- ✨ Happens at **compile time**.
- ✨ Checks if the method exists in the declared type.
- ✨ Does not consider the actual type of the object.

Example: `A obj = new B();` checks if `B` has a method `display()`.

✨ Method Binding

- ✨ Happens at **runtime**.
- ✨ Determines which method implementation to execute based on the actual type of the object.

Example: `obj.display();` executes `B`'s `display()` method if `obj` is an instance of `B`.

11.8. Casting Objects and the `instanceof` Operator

👉Motivation

✨Inheritance allows for polymorphism, where a superclass reference can point to a subclass object.

✨Casting is necessary to access subclass-specific methods or properties.

✨The `instanceof` operator helps ensure safe casting by checking the actual type of the object before casting to prevent runtime errors.

✨There are two types of casting in Java:

- ✨**Implicit Casting (Upcasting)**
- ✨**Explicit Casting (Downcasting)**

👉Implicit Casting (Upcasting)

✨The **upcasting** process automatically converts a subclass reference to a superclass reference.

Syntax:

```
SuperClass object = new SubClass(); // Upcasting (implicit)
```

Explain:

✨No explicit cast is needed; the JVM handles it automatically.

✨Upcasting is always safe because a subclass object is guaranteed to have all the properties and methods of its superclass.

👉Explicit Casting (Downcasting)

✨The **downcasting** process converts a superclass reference back to a subclass reference.

Syntax:

```
SubClass subObject = (SubClass) superObject; // Downcasting (explicit)
```

Explain:

✨Requires explicit cast using parentheses.

✨Must be done carefully to avoid `ClassCastException` at runtime.

Note:

✨Before performing downcasting, use the `instanceof` operator to verify that the object is actually an instance of the target subclass.

👉The `instanceof` Operator

✨The `instanceof` operator checks if an object is an instance of a specific class or subclass.

Syntax:

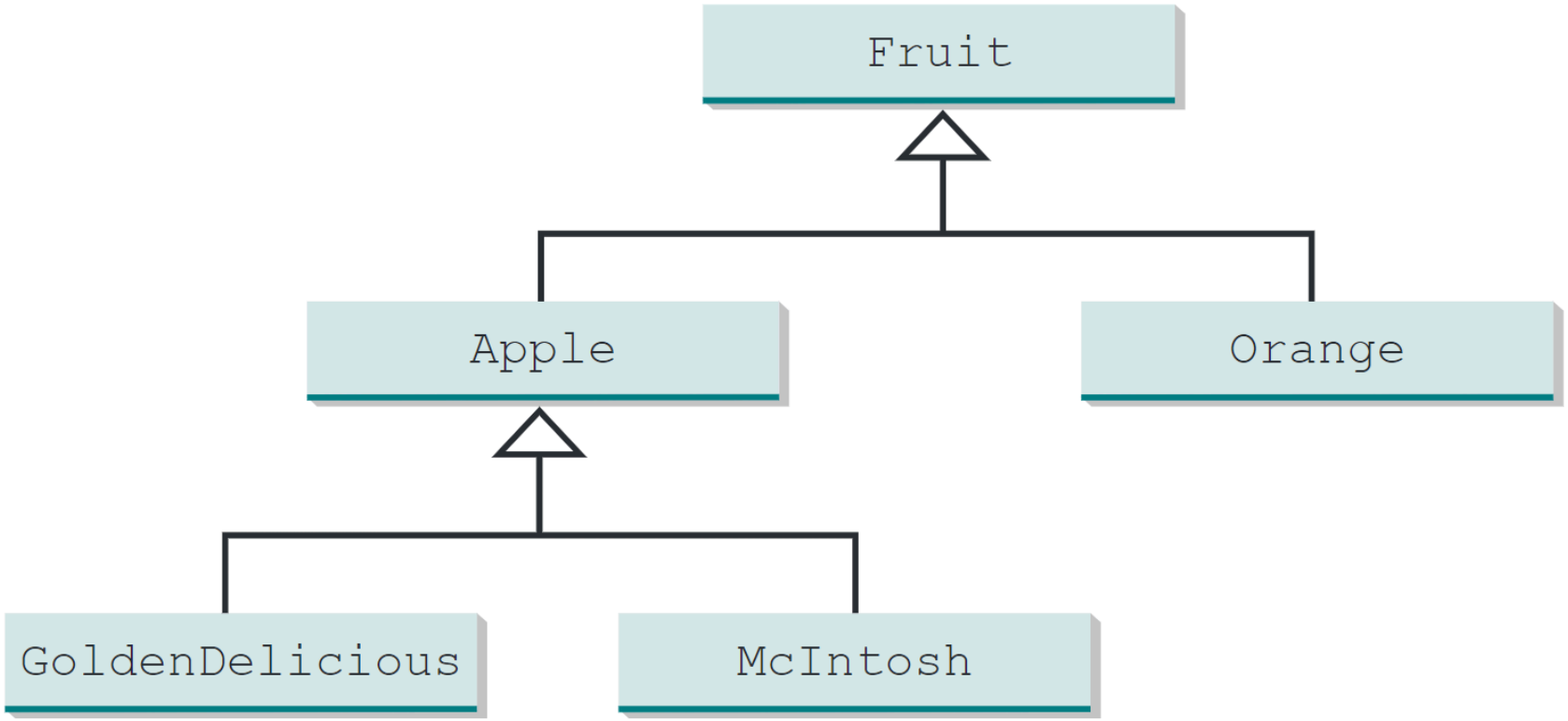
```
if (object instanceof SubClass) {
    // Safe to downcast
}
```

Explain:

✨Returns `true` if the object is an instance of the specified class or subclass, otherwise returns `false`.

Example: Using `instanceof` to Safely Downcast

```
class Fruit {}
class Apple extends Fruit {}
class Orange extends Fruit {}
class GoldenDelicious extends Apple {}
class Macintosh extends Apple {}
```


Fruit
Apple
Orange
GoldenDelicious
McIntosh

```
public class Test {
    public static void main(String[] args) {
        Fruit fruit = new GoldenDelicious();
        if (fruit instanceof Apple) {
            System.out.println("This is an Apple.");
        } else {
            System.out.println("This is not an Apple.");
        }
    }
}
```

Output:

```
This is an Apple.
```

11.9. The Object's `equals` Method

👉Problem of Using `==` for Object Comparison

- ✨The `==` operator checks if two references point to the same object in memory.
- ✨It does not compare the actual contents of the objects.
- ✨This can lead to unexpected results when comparing objects that are logically equal but not the same instance.

Example: Why Integer Objects Can Be Different?

```
Integer a = 128;
Integer b = 128;
System.out.println(a == b); // false, because they are different Integer objects
System.out.println(a.equals(b)); // true, because they have the same value
```

- ✨Therefore, a method `equals()` is needed to compare the contents of objects.

👉The `equals()` Method in Java

✨The `equals()` method is defined in the `Object` class and is used to compare the contents of two objects.

✨By default, it checks for reference equality (like `==`).

Syntax: Default implementation of `equals()` in object.

```
public boolean equals(Object obj) {
    return this == obj; // Default implementation
}
```

👉Overriding the `equals()` Method

✨To compare the contents of objects, you should override the `equals()` method in your class.

✨The overridden method should compare the relevant fields of the objects.

Example: Overriding `equals()` in a `Circle` class

```
class Circle {
    private double radius;
    public Circle(double radius) { this.radius = radius; }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Circle)) return false;
        return Double.compare(radius, ((Circle) obj).radius) == 0;
    }
}
```

Usage:

```
public class Test{
    public static void main(String[] args) {
        Circle c1 = new Circle(5.0);
        Circle c2 = new Circle(5.0);
        System.out.println(c1.equals(c2)); // true, because radii are equal
    }
}
```

Output:

```
true
```

👉 `==` vs `equals()`

Comparison	`==` Operator	`equals()` Method
Purpose	Checks if two references point to the same object (reference equality)	Checks if two objects are logically "equal" (content equality)
Default Behavior	Compares memory addresses	In `Object` class, same as `==` unless overridden
Can Be Overridden?	No	Yes, can be overridden for custom comparison
Usage Example	`a == b`	`a.equals(b)`
Typical Use Case	Primitive types or checking if two references are identical	Comparing object contents (e.g., String, custom classes)

👉Common Mistake

Example: Incorrect Method Signature

Wrong:

```
public boolean equals(Circle c) { ... } // Incorrect!
```

Correct:

```
public boolean equals(Object o) { ... } // Correct!
```

Note:

✨Must use `Object` as the parameter type to properly override `equals()`.

✨This allows the method to accept any object type, enabling polymorphic behavior.

11.10. The `Arrays` Class

👉What is the `Arrays` Class?

✨The `Arrays` class is a utility class in Java that provides methods for manipulating arrays.

✨It is part of the `java.util` package.

```
import java.util.Arrays;
```

👉Features of the `Arrays` Class

Method	Description
`sort(array)`	Sorts the array in ascending order.
`binarySearch(array, key)`	Searches for a value using the binary search algorithm.
`equals(array1, array2)`	Checks if two arrays are equal (element by element).
`fill(array, value)`	Fills the array with the specified value.
`copyOf(array, newLength)`	Copies the array to a new array with the specified length.
`toString(array)`	Returns a string representation of the array.
`asList(array)`	Converts an array to a fixed-size list.
`hashCode(array)`	Returns a hash code based on the contents of the array.
`deepEquals(arr1, arr2)`	Checks equality of nested arrays (multi-dimensional).
`deepToString(array)`	Returns a string for nested arrays (multi-dimensional).

👉Example: Using the `Arrays` Class

```
import java.util.Arrays;
public class ArraysDemo {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 8, 1};
        Arrays.sort(numbers);
        System.out.println("Sorted: " + Arrays.toString(numbers));

        int index = Arrays.binarySearch(numbers, 2);
        System.out.println("Index of 2: " + index);

        int[] copy = Arrays.copyOf(numbers, 6);
        System.out.println("Copied Array: " + Arrays.toString(copy));

        Arrays.fill(copy, 0);
        System.out.println("Filled Array: " + Arrays.toString(copy));
    }
}
```

Output:

```
Sorted: [1, 2, 5, 8]
Index of 2: 1
Copied Array: [1, 2, 5, 8, 0, 0]
Filled Array: [0, 0, 0, 0, 0, 0]
```

11.11. The `List` Methods

👉What are `List` Methods?

✨ `List` methods are built-in utilities in Java for sorting, searching, and manipulating lists.

✨Available as static methods in `Collections` and instance methods in the `List` interface.

✨Work with any `List` implementation (`ArrayList` , `LinkedList` , etc.).

✨Simplify common tasks and improve code readability.

👉Commonly Used Methods in `Collections` Class

Method	Description
`sort(List<T> list)`	Sorts the list in ascending order (natural ordering).
`shuffle(List<?> list)`	Randomly shuffles elements in the list.
`reverse(List<?> list)`	Reverses the order of elements.
`max(Collection<? extends T> coll)`	Returns the largest element.
`min(Collection<? extends T> coll)`	Returns the smallest element.
`binarySearch(List<? extends T> list, T key)`	Searches for an element (must be sorted first).
`fill(List<? super T> list, T obj)`	Replaces all elements with a specified object.
`copy(List<? super T> dest, List<? extends T> src)`	Copies elements from one list to another.

Note:

- ✨ The `<T>` is a **type parameter** that allows the method to work with any type of list.
- ✨ The sign `<?>` is a **wildcard** that allows the method to accept any type of list.
- ✨ The sign `<? super T>` allows the method to accept a list that can hold elements of type `T` or its superclasses.
- ✨ The sign `<? extends T>` allows the method to accept a list that can hold elements of type `T` or its subclasses.

Example: Using `Collections` Methods

```
import java.util.*;
public class CollectionsDemo {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(5, 2, 8, 1);
        Collections.sort(nums);
        System.out.println("Sorted: " + nums);
        Collections.shuffle(nums);
        System.out.println("Shuffled: " + nums);
        System.out.println("Max: " + Collections.max(nums) + ", Min: "
            + Collections.min(nums));
        Collections.reverse(nums);
        System.out.println("Reversed: " + nums);
    }
}
```

Output:

```
Sorted: [1, 2, 5, 8]
Shuffled: [5, 1, 8, 2] (example output, actual order may vary)
Max: 8, Min: 1
Reversed: [2, 8, 1, 5]
```

Explain:

- ✨ `Arrays.asList()` creates a fixed-size list from an array.
- ✨ `Collections.sort()` sorts the list in ascending order.
- ✨ `Collections.shuffle()` randomly rearranges the elements.
- ✨ `Collections.max()` and `Collections.min()` find the largest and smallest elements, respectively.
- ✨ `Collections.reverse()` reverses the order of elements in the list.

11.12. The `ArrayList` Class

👉What is an `ArrayList`?

✨An `ArrayList` is a resizable array implementation of the `List` interface in Java.

✨It allows for dynamic resizing, meaning it can grow or shrink as needed.

✨It is part of the Java Collections Framework and provides a way to store a list of objects.

✨It is implemented in the `java.util` package.

```
import java.util.ArrayList;
```

👉Features of `ArrayList`

Method	Description
`add(E e)`	Appends the specified element to the end of the list.
`remove(Object o)`	Removes the first occurrence of the specified element.
`get(int index)`	Returns the element at the specified position.
`set(int index, E element)`	Replaces the element at the specified position.
`size()`	Returns the number of elements in the list.
`isEmpty()`	Checks if the list is empty.
`clear()`	Removes all elements from the list.
`indexOf(Object o)`	Returns the index of the first occurrence of the element.
`lastIndexOf(Object o)`	Returns the index of the last occurrence of the element.
`toArray()`	Returns an array containing all elements in the list.

👉Declaring and Constructing an `ArrayList`

✨To declare and construct an `ArrayList` object, you need to specify the type of `<E>` elements it will hold using generics.

Syntax:

```
ArrayList<E> list = new ArrayList<>();
```

Explain:

✨ `E` specifies the type of elements the list will hold (such as `String`, `Integer`, etc.).

✨Using `ArrayList<E>` with generics enforces type safety, so only elements of type `E` can be added, reducing runtime errors.

✨The sign `<>` is called the diamond operator, which infers the type from the variable declaration.

Example: Using `ArrayList`

```
import java.util.ArrayList;
public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");
        System.out.println(fruits);           // [Apple, Banana, Cherry]
        System.out.println(fruits.get(0));    // Apple
        fruits.remove("Banana");
        System.out.println(fruits);           // [Apple, Cherry]
        System.out.println(fruits.size());    // 2
    }
}
```

👉 `Arrays` & `ArrayList`

Feature	Array	`ArrayList`
Size	Fixed	Dynamic
Performance	Faster (no resizing)	Slower (due to resizing)
Memory Usage	Efficient	Uses more memory
Flexibility	Cannot grow/shrink	Can grow/shrink dynamically
Type Safety	Not type-safe	Type-safe with generics
Access	Direct access	Indexed access
Methods	Limited methods	Rich set of methods
Iteration	For loop, enhanced	For loop, enhanced, iterator
Usage	Primitive types	Objects (with generics)

👉Visulization of `ArrayList`

java.util.ArrayList<E>	
+ArrayList()	Creates an empty list.
+add(e: E): void	Appends a new element e at the end of this list.
+add(index: int, e: E): void	Adds a new element e at the specified index in this list.
+clear(): void	Removes all elements from this list.
+contains(o: Object): boolean	Returns true if this list contains the element o.
+get(index: int): E	Returns the element from this list at the specified index.
+indexOf(o: Object): int	Returns the index of the first matching element in this list.
+isEmpty(): boolean	Returns true if this list contains no elements.
+lastIndexOf(o: Object): int	Returns the index of the last matching element in this list.
+remove(o: Object): boolean	Removes the first element CDT from this list. Returns true if an element is removed.
+size(): int	Returns the number of elements in this list.
+remove(index: int): E	Removes the element at the specified index. Returns the removed element.
+set(index: int, e: E): E	Sets the element at the specified index.

11.13. Case Study: A Custom Stack Class

Practice.

11.14. The `protected` Data and Methods

👉What is the `protected` Access Modifier?

✨The `protected` keyword allows subclasses and classes in the same package to access data and methods.

✨More accessible than `private`, but more restricted than `public`.

Modifier	Same Class	Same Package	Subclass (Different Package)	Outside Class
`private`	Yes	No	No	No
`protected`	Yes	Yes	Yes	No
`public`	Yes	Yes	Yes	Yes
No Modifier *(default)*	Yes	Yes	No	No

Example: Using `protected` in a Class

```
class A{
    protected int x = 10; // Protected data member
    protected void display() { // Protected method
        System.out.println("Value of x: " + x);
    }
}
class B extends A {
    public void show() {
        display(); // Accessing protected method from superclass
        System.out.println("Accessing protected data: " + x);
    }
}
public class Test {
    public static void main(String[] args) {
        B obj = new B();
        obj.show(); // Accessing protected members through subclass
    }
}
```

Output:

```
Value of x: 10
Accessing protected data: 10
```

Explain:

✨ Class `A` has a protected data member `x` and a protected method `display()`.

✨ Class `B` extends `A` and can access the protected members.

✨ The `show()` method in `B` calls the `display()` method and accesses the protected data member `x`.

✨ In the `main()` method, an instance of `B` is created, and `show()` is called, demonstrating access to protected members.

✨ This shows how `protected` members can be accessed by subclasses, even if they are in different packages, as long as the subclass is defined in the same package or extends the superclass.

11.15. Preventing Extending and Overriding

👉Why Prevent Inheritance and Method Overriding?

- ✨Sometimes, you may want to stop other classes from extending your class or changing how a method works.
- ✨You can do this in Java by using the `final` keyword.
- ✨Making a class or method `final` means no one can extend that class or override that method.
- ✨This helps keep your code safe from unwanted changes and makes sure it works as you expect.
- ✨It's useful when you want to lock down certain parts of your code.

👉Using `final` to Prevent Inheritance

✨The `final` keyword can be used to prevent a class from being extended by other classes.

Syntax:

```
final class MyFinalClass {
    // Class code here
}
```

Explain:

✨The `FinalClass` is declared as `final`, which means it cannot be extended by any other class.

Example: Preventing Inheritance

```
final class FinalClass {
    void display() {
        System.out.println("This is a final class.");
    }
}
class SubClass extends FinalClass { // This will cause a compile-time error
    void show() {
        System.out.println("Trying to extend a final class.");
    }
}
public class Test {
    public static void main(String[] args) {
        FinalClass obj = new FinalClass();
        obj.display();
    }
}
```

👉Using `final` to Prevent Method Overriding

✨The `final` keyword can also be used to prevent a method from being overridden in subclasses.

Syntax:

```
class MyClass {
    final void myMethod() {
        // Method code here
    }
}
```

Explain:

✨The `myMethod()` is declared as `final`, which means it cannot be overridden by any subclass.

Example: Preventing Method Overriding

```
class BaseClass {
    final void display() {
        System.out.println("This method cannot be overridden.");
    }
}
class SubClass extends BaseClass {
    void display() { // This will cause a compile-time error
        System.out.println("Trying to override a final method.");
    }
}
public class Test {
    public static void main(String[] args) {
        BaseClass obj = new BaseClass();
        obj.display();
    }
}
```

End of the Chapter